Documentación técnica del scraper de eventos culturales de Chile

Arquitectura, operación y mantenimiento

Alcance:	extracción, normalización y sincronización del catálogo
Versión documental:	28 de octubre de 2026
Tecnologías principales:	Python, Playwright, NVIDIA NIM, SQLite y Supabase
Ejecución automatizada:	GitHub Actions

Índice

1. Propósito y alcance **3**

1.1. Principios de diseño 3

2. Arquitectura general **3**

2.1. Patrones aplicados 4

3. Estructura del repositorio **4**

4. Fuentes de datos **5**

4.1. Validación de candidatas 6

5. Flujo de procesamiento **6**

5.1. Fase de extracción 6

5.2. Fase de procesamiento y publicación 6

6. Modelo canónico de evento **7**

7. Saneamiento, vigencia y filtro cultural **7**

8. IA, OCR y control de cuota **8**

8.1. Presupuestos y suspensión controlada 8

8.2. Validación de respuestas y diagnóstico 9

9. Estado local, checkpoints y recuperación **9**

10. Exportación y sincronización con Supabase **10**

10.1. Limpieza protegida y conciliación 10

11. Configuración **11**

12. Automatización con GitHub Actions **12**

12.1. Pruebas de base de datos 13

12.2. Cambios del 20 de octubre de 2026 13

12.3. Revisión del 27 de octubre de 2026 13

13. Operación y mantenimiento **13**

13.1. Comandos frecuentes 13

13.2. Cuando una fuente cambia o cae 14

14.Seguridad, límites y consideraciones **14**

15.Documentos relacionados **14**

1. Propósito y alcance

El proyecto construye y mantiene un catálogo de eventos culturales de Chile a partir de fuentes públicas: agendas municipales, espacios culturales, universidades, portales sectoriales y tiqueteras. Su objetivo no es copiar una página completa, sino convertir información pública heterogénea en registros comparables, trazables y vigentes.

El scraper abarca cuatro responsabilidades:

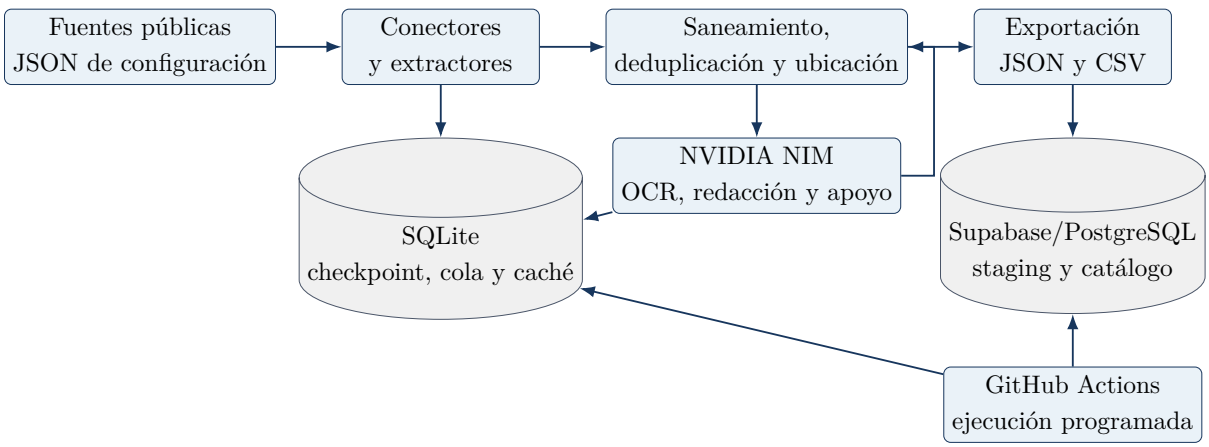
1. descubrir y extraer eventos desde fuentes configuradas;
2. normalizar fechas, ubicación, categorías, organizador y enlaces;
3. enriquecer solo los casos necesarios mediante OCR y redacción asistida por IA;
4. exportar un respaldo local y sincronizar el catálogo publicado con Supabase.

No forma parte de este documento el frontend, la autenticación de usuarios ni el algoritmo de recomendaciones. El scraper alimenta el catálogo que dichas capas consumirán posteriormente.

1.1. Principios de diseño

- **Trazabilidad:** cada evento conserva fuente, URL original, URL oficial cuando existe, método de extracción y momento de captura.
- **Preferencia por datos estructurados:** API, JSON-LD, WordPress REST y tarjetas declarativas se usan antes que OCR o modelos de IA.
- **Mínima presión sobre los sitios:** se respeta el ritmo por dominio, hay caché condicional HTTP y la concurrencia es acotada.
- **Idempotencia:** repetir una ejecución actualiza registros existentes y evita duplicados mediante identificadores estables y deduplicación.
- **Resiliencia:** una fuente caída, un afiche ilegible o una respuesta transitoria de NVIDIA no detienen el resto del proceso.
- **Vigencia:** se retiran eventos vencidos y registros ausentes de fuentes completas, respetando supresiones, interacciones y las protecciones de publicaciones manuales.

2. Arquitectura general



2.1. Patrones aplicados

Patrón	Aplicación
Strategy	Cada conector implementa una estrategia de extracción independiente para una familia de sitios.
Factory/Registry	`ConnectorFactory` selecciona el conector a partir del campo `connector` del JSON de fuente.
Repository	`SourceRepository` carga, valida y entrega la configuración declarativa de `sources/`.
Pipeline	`EventPipeline` orquesta fases en orden, sin acoplar la lógica a una página concreta.
Inyección de dependencias	HTTP, estado, exportador y servicios remotos pueden sustituirse en pruebas.

3. Estructura del repositorio

```
scrape_events.py Punto de entrada de la ejecucion
eventos/ Paquete de extraccion y procesamiento
  connectors/ Estrategias por tipo de fuente
  extractors/ Lectores reutilizables, por ejemplo JSON-LD
  services/ NVIDIA NIM y cliente REST de Supabase
  source_validation/ Pipeline independiente de validacion de fuentes
sources/ Un JSON por fuente y plantillas reutilizables
state/ SQLite local: checkpoint, cola de IA y cache
data/ Exportaciones JSON, CSV, metricas y auditorias
supabase/migrations/ Migraciones SQL versionadas
supabase/tests/ Regresiones SQL con datos ficticios
tools/ Utilidades, entre ellas copia consistente de estado
tests/ Pruebas unitarias y de integracion simulada
.github/workflows/ Automatizacion en GitHub Actions
docs/ Documentacion del scraper
```

Los directorios `state/`, `data/`, cachés y el archivo `.env` son locales y no se publican en Git. Las fuentes, las migraciones, las pruebas y el workflow sí deben versionarse.

4. Fuentes de datos

La configuración vigente contiene 28 fuentes permanentes y una fuente estacional, activa únicamente en septiembre: 29 configuraciones. Chile Cultura, Ticketplus, Passline y PuntoTicket aportan alcance nacional; las demás agregan profundidad territorial o temática.

Fuente	**Cobertura**	**Método principal**
Chile Cultura	Nacional, 16 regiones	API pública paginada
Maipú en Común	Región Metropolitana	API de aplicación mediante Playwright
Centro Cultural GAM	Santiago	JSON-LD
Centro Cultural La Moneda	Santiago	Tarjetas HTML declarativas
Centro Cultural CEINA	Santiago	Tarjetas HTML declarativas
Valpo Cultura	Valparaíso	The Events Calendar API
Espacio Cultural Viña del Mar	Valparaíso	The Events Calendar API
Teatro Municipal de Viña del Mar	Valparaíso	JSON-LD
Agenda Cultural UOH	O'Higgins	WordPress REST
Teatro Biobío	Biobío	WordPress REST y fechas visibles
Agenda Universidad de Talca	Maule	Tarjetas HTML declarativas
Corporación Cultural de Iquique	Tarapacá	Tarjetas institucionales
Centro de Arte Molino Machmar	Los Lagos	Tarjetas institucionales
Museo de la Memoria y los Derechos Humanos	Santiago	API pública Strapi paginada
Teatro Municipal de Chillán	Ñuble	Cartelera y fichas con año explícito
Teatro Regional Cervantes	Los Ríos	Tarjetas con fecha completa
Corporación Cultural de Puerto Montt	Los Lagos	Cartelera móvil y fichas
Teatro Regional del Maule	Talca, Sala TRM	Cartelera y fichas con año explícito
Enoturismo Chile	Interregional	WordPress/EventON
FISA	Varias regiones	Portafolio oficial
Espacio Riesco	Región Metropolitana	Tarjetas HTML declarativas
Parque Cultural de Valparaíso	Valparaíso	WordPress REST
Fundación CorpArtes	Región Metropolitana	Tarjetas HTML declarativas
Cultura Vitacura	Región Metropolitana	Tarjetas HTML declarativas
Santiago Cultura	Santiago	WordPress/EventON incremental
Ticketplus Chile	Nacional, 16 regiones	Selector regional y fichas estructuradas
Passline Chile	Nacional	Búsqueda JSON paginada
PuntoTicket Chile	Nacional	Índice JSON y detalles JSON-LD incrementales
FondasChile	Nacional, septiembre	Tarjetas HTML declarativas

Cada archivo en `sources/` declara URL, región, comuna, organizador, conector, categorías por defecto, límites de paginación y, cuando corresponda, metadatos estáticos de ubicación. Añadir una fuente no exige modificar el archivo principal: primero se intenta configurar un conector existente; solo se crea código nuevo cuando la estructura no coincide con ninguno.

Las cinco incorporaciones del 27 de octubre de 2026 se comprobaron mediante extracción local de solo lectura, sin NVIDIA ni Supabase. Entregaron 43 fichas vigentes antes de deduplicar contra el catálogo: Lago 7, Chillán 5, Cervantes 18, Puerto Montt 4 y Oriente 9. Son observaciones de esa prueba, no conteos permanentes ni una garantía de acceso desde GitHub Actions. Puerto Montt publica una ventana móvil de 30 días: no se afirma cubrir todo su archivo. Chillán utiliza el año explícito de la ficha; Oriente utiliza fechas ISO del JSON-LD y excluye cancelaciones y avisos permanentes.

El 28 de octubre se retiraron Teatro del Lago y Teatro Oriente por respuestas HTTP 403 desde Actions. Los sustituyen Museo de la Memoria y Teatro Regional del Maule, sin aumentar el total de configuraciones. El museo consulta la API pública Strapi utilizada por su cartelera, con fechas ISO UTC. Maule se limita inicialmente a Sala TRM en Talca, lee el año explícito y se detiene antes del archivo de eventos realizados. La prueba local obtuvo cinco y seis fichas vigentes, respectivamente. Maule quedó parcial por fichas sin fecha verificable: no se inventa el año ni se autoriza conciliación de ausencias. El acceso desde el runner sigue pendiente de comprobación.

El conector de tarjetas admite selectores de detalle, campos JSON-LD y enlaces de paginación configurables. Los detalles fallidos temporalmente, la pérdida de campos obligatorios y los ciclos de paginación marcan una extracción parcial, que no autoriza borrados por ausencia. Esta ampliación está documentada en los JSON ejecutables y `docs/SOURCES.md`; no modifica los resultados históricos del inventario Excel.

Las fuentes retiradas se declaran en `settings.json` mediante `retired_source_names`. Durante la siguiente publicación se concilian intencionalmente sus eventos extraídos con procedencia acreditada. Se conservan publicaciones manuales, eventos suprimidos y eventos con interacciones, aunque pertenezcan a esas fuentes.

4.1. Catálogos nacionales JSON

Passline consulta `https://api.passline.com/v1/event/GetBillboardByFilters` mediante POST JSON de lectura: país Chile explícito y páginas de 100, sin un tope artificial de resultados. Usa las fechas completas, recinto, comuna y código regional publicados. No inventa descripciones, direcciones ni nombres de organizador a partir de identificadores numéricos. La prueba local del 28 de octubre obtuvo 1.404 registros vigentes y 1.331 candidatos tras normalización, deduplicación y filtro cultural: 779 aceptados y 552 en revisión. Tres registros inválidos dejaron la fuente parcial, protegiendo la conciliación. Fueron 15 consultas sin NVIDIA.

PuntoTicket consulta `https://www.puntoticket.com/Landing/GetEventosBusqueda` y completa sus fichas con JSON-LD y el bloque propio de descripción. Conserva el intervalo del índice, porque el detalle puede mostrar solo la última función. Los detalles se guardan incrementalmente y se revalidan a las 24 horas o al cambiar el registro JSON. El índice observado contenía 242 entradas: la prueba obtuvo 235 fichas vigentes, 220 después de geografía y deduplicación, y 214 candidatos culturales (70 aceptados y 144 en revisión). Se omitieron siete entradas sin fecha verificable y se completaron 237 peticiones. Se omiten años ausentes y agrupadores sin funciones fechadas.

Ambas fuentes conservan los filtros comunes y la visibilidad administrativa. Los géneros musicales explícitos se normalizan; Club o Fiestas no implican Música. Se excluyen deportes y membresías

de gimnasio sin contenido cultural. La meta aproximada de 1.200–2.000 eventos no obliga a rellenar ni recortar el catálogo. El incremento neto depende de coincidencias y supresiones; debe verificarse tras la sincronización normal. La prueba local no publica datos ni garantiza acceso desde Actions.

Se verifica TLS y se mantiene la pausa por dominio. El User-Agent conserva la identificación honesta del proyecto, normalizada a ASCII por compatibilidad. Ante 403/429 no se rotan identidades ni se elude el bloqueo. La extracción no consume NVIDIA; el enriquecimiento posterior mantiene sus colas, presupuestos y límite global de 40 RPM. La segunda pasada de PuntoTicket reutilizó sus detalles y necesitó solo dos peticiones. Pasaron 239 pruebas de Python, incluidas paginación, fechas estrictas, caché, bloqueo y filtros; la verificación no publicó nuevos eventos en Supabase.

4.2. Validación de candidatas

Antes de integrar una fuente, el comando `validar_fuentes.bat` revisa el inventario de candidatas sin extraer eventos ni modificar Supabase. Comprueba disponibilidad HTTP, `robots.txt`, presencia de señales de agenda, fechas, enlaces, pertinencia cultural y cobertura chilena. Sus resultados se guardan en `data/source_validation/`; una aprobación técnica no habilita una fuente automáticamente, pues la configuración y las pruebas siguen siendo una decisión del equipo.

5. Flujo de procesamiento

5.1. Fase de extracción

1. Se cargan las fuentes habilitadas para el mes actual; FondasChile se omite fuera de septiembre.
2. Las fuentes se agrupan por dominio. Hasta cuatro dominios se procesan en paralelo, pero cada dominio conserva su propio ritmo.
3. Cada conector devuelve objetos `Event` con un esquema común.
4. Se aplican limpieza inicial, deduplicación interna y validación de disponibilidad de fichas.
5. Cada fuente exitosa o vacía de forma válida queda registrada en un checkpoint local.

La fase puede ejecutarse sin IA ni Supabase:

```
python scrape_events.py --phase extract --local-only
```

5.2. Fase de procesamiento y publicación

1. Se recupera la extracción compatible más reciente desde SQLite.
2. Se sanea el texto globalmente y se deduplican eventos entre fuentes.
3. Se aplican valores de ubicación declarados por fuente y reglas deterministas.
4. Si Supabase está habilitado, se detectan eventos nuevos, modificados, suprimidos o pertenecientes a organizadores bloqueados.
5. Solo el subconjunto pendiente pasa por OCR, redacción y enriquecimiento de ubicación.
6. Se ejecuta el filtro cultural, se descartan fechas inválidas o vencidas y se exportan JSON/CSV.

7. Se publica de forma atómica en Supabase y se reconcilian registros obsoletos de fuentes completas.

```
python scrape_events.py --phase process --supabase
```

La separación de fases permite conservar el trabajo de extracción si se agota una cuota de IA o falla temporalmente el destino remoto.

6. Modelo canónico de evento

Todo conector entrega el mismo modelo lógico. Los campos fundamentales son `title`, `start_date`, `end_date`, `venue`, `location`, `commune`, `region`, `organizer`, `categories`, `source_url` y `official_url`.

Las categorías son siempre una lista de disciplinas culturales, no una cadena. Un evento puede pertenecer simultáneamente a música y danza. Familiar es un público; gratuito es una condición de acceso, no una disciplina. En JSON se guarda como arreglo; en CSV se serializa como JSON dentro de una celda; en Supabase se normaliza mediante `categories` y `event_categories`.

La revisión del 27 de octubre incorpora 18 disciplinas canónicas y el objeto `classification`: versión, etiquetas originales, formatos, públicos, programas, evidencia y decisión. Se recalcula incluso para eventos conocidos y checkpoints; cambiar la taxonomía no obliga a repetir OCR ni redacción. Guías de librerías y fichas de espacios no se convierten en eventos por declarar una vigencia de varios años. Una duración larga, por sí sola, no descarta una programación real.

La revisión de reglas del 28 de octubre se identifica independientemente del esquema mediante `rules_revision` e invalida solamente las evidencias de clasificación incompatibles. Ticketplus completa el JSON-LD con la descripción propia de la ficha y sus etiquetas Disciplina/Género, sin tomar texto de relacionados ni efectuar otra solicitud HTTP. Se conservan los controles contra deportes, servicios y directorios. Se reconocen además evidencias explícitas como canto, collage e instalación sonora. No requiere una nueva migración SQL.

La ubicación se resuelve de forma transparente y cada evento incluye:

- `location`: texto canónico que nunca queda vacío;
- `location_precision`: dirección, recinto, comuna, región o modalidad en línea;
- `location_source`: dato publicado, configuración de fuente, regla determinista, IA o fallback geográfico.

No se inventan direcciones. Cuando la evidencia solo permite identificar una comuna o región, se conserva exactamente ese nivel y se marca la menor precisión.

7. Saneamiento, vigencia y filtro cultural

El saneador común elimina HTML residual, reglas CSS, scripts, shortcodes de WordPress/WPBakery y descripciones desproporcionadas. Esto evita que cambios visuales de un sitio se conviertan en contenido del catálogo.

Un evento se conserva solo si tiene fecha de inicio válida y todavía está vigente. La validación de URL retira fichas confirmadas con HTTP 404/410 o páginas de error inequívocas; ante timeout, bloqueo temporal o error 5xx el registro se conserva para no borrar información por una falla pasajera.

El filtro realiza una comprobación antes de consumir IA y otra después del enriquecimiento. Tiene tres resultados: *aceptado*, *revisión* y *excluido*. Se exige contenido cultural verificable para aceptar; el nombre de la fuente, organizador o recinto no basta. La política activa es `cultural_review_action=keep`: los candidatos ambiguos vigentes pueden publicarse, conservando `classification.decision=review`. No se convierten en aceptados ni reciben categorías inventadas. La alternativa `quarantine` los conserva en `draft`. Los directorios y demás exclusiones siguen fuera. La música de acompañamiento no convierte una clase de fitness en concierto. La auditoría queda en `data/cultural_filter_audit.json`.

Ticketplus y las demás fuentes usan primero reglas sobre el contenido. Solo los casos aún ambiguos consultan NVIDIA, con categorías permitidas y evidencia literal: lotes de 10, hasta 100 candidatos por corrida, cinco minutos adicionales de presupuesto y suspensión tras tres fallos consecutivos. Se mantiene el límite global de 40 RPM y se reutiliza evidencia compatible local o remota. Un fallo no inventa la categoría Otros ni impide sincronizar los pendientes según la política de publicación activa.

Esta versión requiere la migración `20261027000100_cultural_classification.sql`. Instalarla no altera filas del catálogo. La publicación atómica conserva los metadatos y archiva exclusiones ya existentes sin borrar interacciones. El comando `python -m tools.reclassify_catalog` simula el mantenimiento sin NVIDIA; solo `-apply` habilita escritura después de revisar la propuesta. Se protegen eventos manuales, supresiones, fuentes inactivas y organizadores bloqueados. El diccionario de base de datos documenta el campo privado `event_provenance.classification` y la RPC de mantenimiento.

La herramienta de mantenimiento conserva su política restrictiva: los casos `review` quedan como borradores. No se utiliza `-apply` para hacer visibles esos casos. La sincronización normal es la que aplica `keep` a los eventos extraídos, sin cambiar indiscriminadamente borradores manuales.

Para el catálogo existente se aplicó la migración `20261028000100_publish_existing_reviews.sql`. Publicó 369 borradores automáticos vigentes conservando `decision=review`, las categorías y la evidencia. El conteo verificado quedó en 946 publicados, dos borradores y 289 archivados, con los mismos 1.237 registros. No modificó tablas ni permisos, no restauró archivados/cancelados y respetó manuales, supresiones, organizadores bloqueados y fuentes inactivas/retiradas. Las comprobaciones de vigencia usan el reloj del servidor y el día de Santiago. Las cifras son una observación, no una cantidad fija esperada.

7.1. Clasificación y visibilidad administrativa

La clasificación automática no equivale a visibilidad. `event_provenance.classification.decision` mantiene aceptado, revisión o excluido; `events.status` mantiene el ciclo de publicación. La columna existente `events.is_suppressed` controla una ocultación administrativa persistente: con valor verdadero no se muestra el evento y el scraper no restablece esa columna. Desmarcarla no cambia la clasificación ni elimina los requisitos de vigencia y publicación.

El panel debe filtrar por clasificación y visibilidad y ofrecer acciones independientes para ocultar o volver a mostrar. Su backend debe comprobar el rol `admin` antes de leer la procedencia privada o modificar esa marca; no se entregan permisos de moderación a `visitor` ni una clave secreta al navegador. Esta revisión prepara el contrato del scraper y verifica la persistencia de la supresión; no implementa ni valida la interfaz web del panel y no amplía las políticas RLS.

8. IA, OCR y control de cuota

NVIDIA NIM se usa como respaldo, no como requisito de extracción. Las fichas de fuentes con API, JSON-LD, WordPress REST o tarjetas declarativas se leen sin IA, pero sus eventos nuevos o modificados pueden necesitar enriquecimiento posterior. La IA interviene en estos casos:

- respaldo de extracción semántica cuando un conector genérico no encuentra datos estructurados y tiene habilitado ese respaldo;
- OCR de afiches con imagen cuyo resultado está ausente, pendiente o vencido en caché; no depende de que la descripción HTML esté vacía;
- redacción editorial de texto público, preservando el texto de origen para trazabilidad;
- resolución por lotes de ubicaciones incompletas;
- clasificación con evidencia de disciplinas aún ambiguas, sin repetir operaciones ya resueltas.

El límite global se configura en 40 solicitudes por minuto. Todas las llamadas, incluso las iniciadas por fuentes concurrentes, pasan por el mismo regulador. Las respuestas 429, `Retry-After` y secuencias de timeouts activan pausas controladas; los pendientes no se pierden, sino que pasan a una cola SQLite para una ejecución futura.

8.1. Presupuestos y suspensión controlada

El timeout de red no es el presupuesto de toda una fase. El OCR tiene, por defecto, 45 segundos de timeout por solicitud, un reintento y hasta 100 segundos de presupuesto por operación. La etapa admite hasta 100 imágenes, pero dispone además de un presupuesto acumulado de 20 minutos. La redacción dispone de otros 20 minutos y la ubicación de 5 minutos. El tiempo restante recorta la espera y los reintentos de la siguiente operación.

Cada etapa se suspende después de tres operaciones consecutivas sin resultado válido. Un resultado válido reinicia el contador; no se cuenta simplemente el total de errores de toda la corrida. En redacción y ubicación, un lote con resultados válidos parciales conserva los completados y deja el resto pendiente. La suspensión no desactiva automáticamente todas las etapas de IA ni descarta eventos que ya sean válidos.

Los presupuestos suman 45 minutos nominales y reservan margen dentro de los 80 minutos del paso de procesamiento en Actions. No son una garantía estricta de duración ante bloqueos de red o de Supabase. El límite RPM sigue vigente; no se alternan cuentas ni claves para aumentarlo. Las respuestas HTTP 401, 403 y 410 constituyen una indisponibilidad permanente para esa ejecución y desactivan el respaldo IA global, a diferencia de la suspensión de una sola etapa.

8.2. Validación de respuestas y diagnóstico

Las solicitudes piden una respuesta sin streaming. Se comprueban por separado la envoltura JSON HTTP, el texto devuelto por el modelo y los campos requeridos. En OCR y redacción se admiten cercos Markdown, preámbulos y una lista unitaria que contenga el objeto esperado; no se inventan campos ni se recuperan subobjetos de estructuras truncadas. Una respuesta marcada como incompleta o filtrada se rechaza antes de actualizar el evento.

Los errores de envoltura informan estado HTTP, tipo de contenido y tamaño, sin imprimir el cuerpo ni las credenciales. Un `Read timed out` no demuestra agotamiento de cuota: la evidencia de límite RPM es HTTP 429. La URL cultural identifica el evento afectado; el host del error identifica el servicio con el que falló la comunicación.

El mensaje de suspensión del OCR indica el resto del lote seleccionado, no el tamaño total de la cola. Una publicación correcta en Supabase puede coexistir con enriquecimientos pendientes. La suspensión permite continuar hasta la sincronización, pero no convierte un fallo real de Supabase en un éxito ni garantiza la calidad de información que no pudo leerse.

Antes de usar IA, Supabase compara URL original y una clave combinada de título, fecha y lugar. Si el evento ya existe y no cambió, se reutilizan OCR y texto editorial previamente almacenados. Esta deduplicación reduce costo, tiempo y consumo de cuota.

La RPC privada `match_existing_events(jsonb)` busca por clave externa, URL de extracción o URL oficial. Desde la revisión del 20 de octubre utiliza dos índices B-tree de URL normalizada y ramas de igualdad separadas. Conserva la coincidencia más reciente por clave local y resuelve empates por identificador. El cliente divide las consultas en lotes de entre 1 y 500 candidatos; el workflow utiliza 200. Esto evita respuestas masivas que puedan truncarse por el límite de filas de PostgREST, sin aumentar el límite de NVIDIA.

9. Estado local, checkpoints y recuperación

El archivo `state/pipeline.sqlite3` conserva datos operativos, no el catálogo público. Incluye checkpoints por fuente, una extracción consolidada, cola de enriquecimiento, caché de resultados de IA y de detalles, y el enfriamiento pendiente de NVIDIA. La caché HTTP condicional se almacena separadamente en `.cache/http/`; no debe confundirse con la cola SQLite.

En GitHub Actions, el estado se guarda como artefacto después de la extracción y nuevamente al finalizar. Una ejecución siguiente restaura el último artefacto disponible. Por ello, si falla la fase de enriquecimiento, no se requiere volver a recorrer todas las páginas: la siguiente corrida retoma los eventos y pendientes conservados.

Las tareas pueden estar pendientes, en proceso, completadas o con error reintentable. Las completadas se reutilizan mientras su entrada y caché sigan siendo compatibles. Una corrida terminada inicia un nuevo ciclo de revisión en la siguiente ejecución; no congela indefinidamente el listado de fuentes. Cambiar los JSON, como al incorporar las cinco fuentes nuevas, invalida la compatibilidad de la extracción consolidada anterior: debe ejecutarse primero la fase de extracción. El workflow ya sigue ese orden. El OCR pendiente solo puede retomarse si se conserva/restaura el estado y el evento sigue siendo elegible.

No deben ejecutarse simultáneamente dos procesos que compartan el mismo estado local o las mismas credenciales de IA. El workflow usa un grupo de concurrencia para impedir que dos ejecuciones programadas se solapen.

10. Exportación y sincronización con Supabase

El respaldo local se escribe en `data/events.json`, `data/events.csv`, archivos separados por región, `data/source_metrics.json` y `data/run_summary.json`. Al final de cada corrida se informa un total consolidado y duración completa.

La sincronización remota usa la API REST de Supabase con una clave secreta exclusiva del proceso de ingesta. El flujo remoto es:

1. transformar eventos en entidades normalizadas: fuentes, organizadores, comunas, categorías, eventos, procedencia y medios;
2. cargar fragmentos en la tabla privada `catalog_staging`;
3. ejecutar `publish_staged_catalog(...)` para publicar todo el lote dentro de una transacción;
4. conciliar ausencias e invocar `cleanup_expired_events()` dentro de esa misma transacción, si la limpieza está habilitada;
5. registrar el resultado de sincronización en `scrape_runs`.

Las consultas y cargas REST idempotentes reintentan fallas transitorias de red, timeout o disponibilidad temporal. La RPC de publicación no se repite automáticamente si su respuesta se pierde, porque podría haber finalizado y eliminado el staging; así se evita una publicación ambigua o duplicada.

Los organizadores bloqueados y los eventos suprimidos individualmente se excluyen antes de OCR, redacción y publicación. De este modo, una solicitud de retiro no se revierte en la siguiente ejecución.

10.1. Limpieza protegida y conciliación

`cleanup_expired_events()` es una función PostgreSQL privada que devuelve un `bigint`; no es un trigger ni un procedimiento invocable con `CALL`. Usa `now()` del servidor al inicio de la transacción. Evalúa tanto eventos manuales como extraídos:

- con `end_at`, elimina cuando el término es estrictamente anterior al reloj del servidor;
- sin `end_at`, elimina cuando el inicio es anterior al comienzo del día actual en `America/Santiago`; conserva todo el día local de inicio;
- conserva eventos suprimidos o con cualquier interacción de usuario, sea positiva o negativa;
- conserva fechas sin inicio o con término anterior al inicio, para revisión;
- elimina por cascada las relaciones de categorías, medios y procedencia del evento retirado.

La conciliación por ausencia es otra regla: requiere una fuente autorizada, una fila no vista en la sincronización actual, procedencia en `event_provenance` y un método distinto de `manual`. También protege supresiones e interacciones. Puede retirar una ficha con fecha futura si ya no aparece en una fuente completa; una fuente fallida o parcial no autoriza esa conciliación.

Las dos rutas utilizan un bloqueo `SHARE` de `user_event_interactions` para evitar perder señales concurrentes; el tiempo de espera de bloqueos se limita a cinco segundos. Una falla revierte publicación, relaciones y borrados, manteniendo staging. Las inserciones de interacciones pueden esperar durante esta transacción.

La publicación devuelve `removed_stale` y `removed_expired`. El exportador guarda esos contadores en `scrape_runs.metadata` cuando marca el éxito. No son el total restante del catálogo. La escritura final de la bitácora ocurre después de la RPC y no revierte una publicación ya confirmada si falla.

La conservación de eventos con interacciones es temporal, hasta desacoplar las señales del recomendador. El frontend debe seguir filtrando las fechas: una fila conservada por protección no necesariamente representa un evento vigente. Las RPC de limpieza y publicación solo admiten ejecución con `service_role`; `anon` y `authenticated` no tienen ese permiso.

11. Configuración

La configuración sensible vive en `.env` y nunca debe subirse al repositorio. Las variables mínimas para sincronizar son:

```
SUPABASE_ENABLED=true
SUPABASE_URL=https://TU_PROYECTO.supabase.co
SUPABASE_SECRET_KEY=sb_secret_...
NVIDIA_API_KEY=...
```

Los parámetros operativos más relevantes son:

Variable	Finalidad
`SOURCE_WORKERS`	Dominios procesados en paralelo; valor recomendado: 4.
`HTTP_CACHE_ENABLED`	Activa revalidación condicional mediante ETag y Last-Modified.
`NVIDIA_RPM_LIMIT`	Límite global de solicitudes a NVIDIA; configurado en 40.
`NVIDIA_OCR_MAX_IMAGES`	Máximo de afiches OCR por ejecución.
`NVIDIA_OCR_STAGE_BUDGET_SECONDS`	Presupuesto acumulado OCR; 1200 segundos (20 minutos).
`NVIDIA_REWRITE_STAGE_BUDGET_SECONDS`	Presupuesto acumulado de redacción; 1200 segundos.
`NVIDIA_LOCATION_STAGE_BUDGET_SECONDS`	Presupuesto acumulado de ubicación; 300 segundos.
`NVIDIA_OCR_FAILURE_THRESHOLD`	Corte tras operaciones OCR fallidas consecutivas; 3 por defecto.
`NVIDIA_REWRITE_FAILURE_THRESHOLD`	Umbral equivalente para lotes de redacción; 3 por defecto.

Variable	Finalidad
NVIDIA_LOCATION_FAILURE_THRESHOLD	Umbral equivalente para lotes de ubicación; 3 por defecto.
NVIDIA_REWRITE_ENABLED	Activa o desactiva la redacción editorial.
SUPABASE_DELETE_EXPIRED	Elimina eventos ya terminados al sincronizar.
SUPABASE_RECONCILE_STALE	Retira eventos no vistos por fuentes completas.
SUPABASE_BATCH_SIZE	Fragmentos REST; 200 en el workflow. La consulta de coincidencias limita sus lotes a 500 como máximo.
SUPABASE_MAX_RETRIES	Reintentos de operaciones REST idempotentes ante fallas transitorias.
SUPABASE_RETRY_MAX_WAIT_SECONDS	Espera máxima entre reintentos REST.

12. Automatización con GitHub Actions

El workflow separa validación y ejecución en dos trabajos sobre `ubuntu-24.04`. El trabajo `validate` instala dependencias, ejecuta las pruebas de Python y utiliza PostgreSQL 17 temporal para aplicar todas las migraciones y probar las regresiones SQL. El contenedor existe únicamente en GitHub Actions: no requiere instalar Docker en el computador local ni contenerizar el scraper.

El trabajo `scrape` depende del éxito de `validate`. Instala Chromium, restaura checkpoints, extrae, conserva un artefacto, enriquece y sincroniza con limpieza protegida. Conserva las copias de estado al finalizar aunque una fase falle. Las credenciales se almacenan como *Repository secrets*: `SUPABASE_URL`, `SUPABASE_SECRET_KEY` y `NVIDIA_API_KEY`. La validación SQL no utiliza estas claves.

El resumen de duración pertenece al comando que lo genera. Como extracción y procesamiento se ejecutan por separado, el tiempo de una fase no equivale al tiempo completo del workflow. El total de eventos sincronizados tampoco representa necesariamente todas las filas existentes en Supabase: pueden conservarse publicaciones manuales, supresiones o registros protegidos por interacciones.

La programación vigente es diaria a las 23:37 en la zona `America/Santiago`, mediante `37 23 * * *`. El workflow conserva `workflow_dispatch` para ejecución manual. La zona horaria evita ajustar el cron por horario de verano; el programador de GitHub puede retrasar el inicio. Las migraciones remotas se aplican por separado y no en cada ejecución programada.

12.1. Pruebas de base de datos

`tools/test_database.sh` exige una base vacía llamada `eventos_test`, instala un contrato mínimo ficticio de Auth y aplica las migraciones ordenadas antes de ejecutar `supabase/tests/*.sql`. No lee `.env` y nunca debe apuntar al proyecto de producción. El fixture comprueba el comportamiento del catálogo, no el servicio real de autenticación.

Las regresiones cubren vencimiento, día chileno, fechas incoherentes, supresiones, interacciones, cascadas, permisos, idempotencia, coincidencias por clave/URL y rollback de publicación ante un

fallo forzado. El 20 de octubre pasaron las 144 pruebas de Python, incluida la comprobación de cobertura documental de migraciones, y las tres suites SQL locales sobre PostgreSQL embebido con PGlite, en UTC, Santiago y Tokio. La ejecución del nuevo trabajo con PostgreSQL 17 en Actions se verifica después de subir el cambio; no se presenta como una prueba remota ya realizada.

12.2. Cambios del 20 de octubre de 2026

- `20261020000100`: crea `cleanup_expired_events()` y restringe su ejecución al proceso de ingesta.
- `20261020000200`: incorpora índices para URL de origen y oficial. Optimiza `match_existing_events(jsonb)` sin modificar su firma.
- `20261020000300`: conecta la limpieza protegida con la publicación atómica y protege las publicaciones manuales frente a la conciliación.

En esa revisión del 20 de octubre, las nueve migraciones del scraper quedaron sincronizadas. Su instalación no ejecutó el scraper ni eliminó eventos. Esa revisión no añadió tablas, columnas ni relaciones; no describe migraciones posteriores de autenticación ajenas al alcance de este manual.

12.3. Revisión del 27 de octubre de 2026

Se incorporaron las cinco fuentes descritas en la sección de fuentes y se reforzó el conector de tarjetas con lectura de detalles, fechas completas, JSON-LD y paginación segura. Se mantuvieron los checkpoints y los presupuestos por etapa de IA. Las 178 pruebas de Python pasaron localmente, incluidos los casos de suspensión OCR, continuidad hacia el exportador simulado, recuperación de pendientes, estructuras de las nuevas agendas y coherencia documental de fuentes y presupuestos. La prueba real de las cinco fuentes no utilizó NVIDIA ni sincronizó Supabase. No se afirma una ejecución integral nueva en producción por haber pasado estas pruebas.

12.4. Revisión del 28 de octubre de 2026

Una consulta de solo lectura encontró 1.237 registros en Supabase: 577 publicados, 371 borradores y 289 archivados; 297 borradores procedían de Ticketplus. La ejecución automática había terminado correctamente. La reducción visible no demostraba una pérdida de todos esos registros, sino el efecto de la clasificación y la cuarentena. No se publican todos los borradores en bloque.

Se corrigieron los reemplazos de fuentes, la lectura del texto original de Ticketplus y la clasificación determinista. El resumen diferencia publicables de borradores e identifica las revisiones visibles. Se activó `keep` conservando la distinción de clasificación y el control administrativo de visibilidad. Pasaron 225 pruebas locales de Python, incluidas las nuevas regresiones de supresión persistente. La validación de fuentes no consumió NVIDIA ni modificó Supabase. Para renovar la evidencia de Ticketplus debe ejecutarse también la fase de extracción: procesar solamente un archivo antiguo no obtiene su descripción completa. Su cambio de configuración invalida sus checkpoints, sin borrar las colas de las otras fuentes. La siguiente sincronización normal podrá publicar los candidatos vigentes aceptados o en revisión que no estén suprimidos ni bloqueados; el conteo final no se garantiza por adelantado.

13. Operación y mantenimiento

13.1. Comandos frecuentes

```
# Instalacion en Windows
instalar.bat

# Ejecucion completa local
ejecutar.bat

# Pruebas
python -m unittest discover -s tests -v

# Solo extraccion, sin IA ni Supabase
python scrape_events.py --phase extract --local-only

# Procesar un checkpoint y sincronizar
python scrape_events.py --phase process --supabase

# Aplicar migraciones de Supabase en Windows
migrar_supabase.bat
```

13.2. Cuando una fuente cambia o cae

1. revisar el resumen de fuente y confirmar si falló, quedó parcial o está legítimamente vacía;
2. comprobar la página pública y sus términos de uso antes de cambiar la extracción;
3. preferir API, JSON-LD o endpoint público sobre selectores visuales frágiles;
4. actualizar solo el JSON de fuente cuando un conector declarativo sea suficiente;
5. agregar o corregir pruebas con una muestra representativa antes de habilitar la fuente;
6. si la fuente falla, mantenerla fuera de la reconciliación: el pipeline no borra su catálogo por una extracción parcial o fallida.

La guía operativa detallada está en `sources/GUIA_AGREGAR_Y_MANTENER_FUENTES.md`.

14. Seguridad, límites y consideraciones

Se consultan únicamente recursos públicos. Antes de añadir una fuente se deben revisar términos de uso, `robots.txt`, condiciones de reutilización y frecuencia razonable de consulta. El sistema no debe intentar evadir bloqueos ni acceder a contenido privado.

Las claves secretas de Supabase se usan solamente por el scraper o GitHub Actions; nunca deben llegar al frontend, a archivos versionados ni a logs. El OCR y la redacción automatizada no sustituyen una revisión editorial ni garantizan, por sí solos, conformidad con políticas de terceros.

15. Documentos relacionados

- `README.md`: instalación, comandos cotidianos y visión rápida.

- `docs/SOURCES.md`: procedencia y tratamiento de cada fuente.
- `docs/DATABASE.md`: diccionario de la base de datos, mantenido por separado.
- `sources/GUIA_AGREGAR_Y_MANTENER_FUENTES.md`: operación detallada de fuentes.
- `docs/ESTANDAR_DOCUMENTACION.md`: formato común para documentos y diagramas del proyecto.
- `docs/historico/`: auditorías y validaciones fechadas como evidencia.

Fin de la documentación técnica del scraper.